

TJSC 2026 - Analista de Sistemas

Banco de Dados e Engenharia de Dados

Aula 5

SQL parte 2 - DDL, DML, DCL, TCL, constraints, views, triggers e stored procedures

Material original, calibrado para o edital TJSC 2026 e para o estilo FGV. Foco em distinções técnicas, leitura de comandos SQL, cenários corporativos e questões autorais com distratores plausíveis.

Disciplina	Banco de Dados e Engenharia de Dados
Aula da trilha	Aula 5 de 12
Recorte	SQL parte 2: DDL, DML, DCL, TCL, constraints, views, triggers e stored procedures
Edital	Linguagem SQL, SGBDs e administração de banco de dados
Estratégia	Distinguir comando, objeto, efeito transacional, permissão e consequência prática

1. Identificação da aula e recorte do edital

Esta é a Aula 5 da trilha de Banco de Dados e Engenharia de Dados para o TJSC 2026 - Analista de Sistemas, banca FGV. O recorte é SQL parte 2: comandos de definição, manipulação, controle de acesso, controle de transações, restrições de integridade, views, triggers e stored procedures.

A aula preserva o tópico do edital relativo à Linguagem SQL e prepara a ponte para a Aula 6, que tratará de transações, ACID - Atomicity, Consistency, Isolation and Durability -, concorrência, isolamento, locks, deadlock, commit, rollback, backup e recuperação.

Foco FGV: a banca tende a cobrar SQL em contexto de sistema: um comando, uma tabela, uma view, uma procedure ou uma regra de integridade é apresentada, e o candidato deve perceber o efeito correto. Em vez de decorar apenas listas, você precisa saber a consequência prática de cada comando.

2. Mapa da aula

SQL - Structured Query Language

```
-- DDL - Data Definition Language: CREATE, ALTER, DROP, TRUNCATE
-- DML - Data Manipulation Language: INSERT, UPDATE, DELETE, MERGE
-- DCL - Data Control Language: GRANT, REVOKE
-- TCL - Transaction Control Language: COMMIT, ROLLBACK, SAVEPOINT
-- Constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT
-- Views: view comum, view materializada, view de segurança
-- Triggers: BEFORE, AFTER, INSTEAD OF; linha ou comando; DML/DDL/logon conforme SGBD
-- Stored procedures: rotina armazenada, parâmetros, encapsulamento de lógica
```

3. Siglas e vocabulário de prova

Sigla	Expansão	Uso em prova
SQL	Structured Query Language	Linguagem estruturada para consultar, definir, manipular e controlar dados em bancos relacionais.
DDL	Data Definition Language	Define estrutura: CREATE, ALTER, DROP, TRUNCATE.
DML	Data Manipulation Language	Manipula dados: INSERT, UPDATE, DELETE, MERGE.
DCL	Data Control Language	Controla privilégios: GRANT e REVOKE.
TCL	Transaction Control Language	Controla transações: COMMIT, ROLLBACK e SAVEPOINT.
PK	Primary Key	Restrição de unicidade e não nulidade para identificar tuplas.
FK	Foreign Key	Restrição que mantém referência válida para outra tabela.
CRUD	Create, Read, Update, Delete	Operações de aplicação: criar, consultar, atualizar e excluir dados.
SGBD	Sistema de Gerenciamento de Banco de Dados	Software que gerencia armazenamento, consulta, integridade, segurança e transações.

Pegadinha de edital: algumas classificações usam DQL - Data Query Language - para SELECT. No edital e em muitas questões de concurso, a divisão mais cobrada é DDL, DML, DCL e TCL. Se aparecer uma categoria inventada como DRL - Data Recovery Language - desconfie.

4. DDL - Data Definition Language

DDL - Data Definition Language - é o grupo de comandos SQL usado para definir ou modificar a estrutura dos objetos do banco de dados. Em prova, a palavra-chave é estrutura: tabela, coluna, esquema, constraint, índice, view e outros objetos.

Comando	Função essencial	Exemplo de uso
CREATE	Cria objeto de banco.	CREATE TABLE processo (...)
ALTER	Altera estrutura de objeto existente.	ALTER TABLE processo ADD COLUMN segredo BOOLEAN
DROP	Remove objeto de banco.	DROP TABLE processo
TRUNCATE	Remove rapidamente as linhas de uma tabela, preservando a estrutura.	TRUNCATE TABLE log_acesso

4.1 CREATE TABLE e a estrutura mínima

```
CREATE TABLE processo (
  id_processo      INTEGER PRIMARY KEY,
  numero          VARCHAR(30) NOT NULL UNIQUE,
  classe          VARCHAR(80) NOT NULL,
  data_distribuicao DATE NOT NULL,
  segredo_justica BOOLEAN DEFAULT FALSE
);
```

O comando acima cria a tabela e define restrições. A FGV pode cobrar se PRIMARY KEY, UNIQUE, NOT NULL e DEFAULT pertencem ao desenho estrutural da tabela, e não a uma consulta analítica.

4.2 ALTER TABLE: mudança incremental

```
ALTER TABLE processo
ADD COLUMN data_baixa DATE;

ALTER TABLE processo
ADD CONSTRAINT ck_datas
CHECK (data_baixa IS NULL OR data_baixa >= data_distribuicao);
```

ALTER TABLE altera a definição de uma tabela já existente. Em sistemas corporativos com Flyway ou Liquibase, esse tipo de comando aparece em scripts de migração de banco.

4.3 DROP, DELETE e TRUNCATE: a tríade que derruba candidato

Comando	Categoria	Remove linhas?	Remove estrutura?	Filtro WHERE?
DELETE FROM tabela WHERE ...	DML	Sim, linhas selecionadas	Não	Sim
TRUNCATE TABLE tabela	DDL em muitos SGBDs	Sim, geralmente todas	Não	Não
DROP TABLE tabela	DDL	Sim, por remover a tabela	Sim	Não

Regra de prova: DELETE remove registros; TRUNCATE esvazia a tabela de modo estrutural/administrativo; DROP remove o objeto. Se o enunciado fala em manter a estrutura, DROP normalmente está errado.

5. DDL em migrações corporativas

No seu contexto de Java, Quarkus, APIs e sistemas corporativos, DDL aparece sobretudo em scripts de migração: criação de tabelas, alteração de colunas, inclusão de constraints, criação de índices e views. Em ambientes com esteira corporativa, o comando passa por versionamento, revisão e deploy controlado.

Exemplo de migração V12__cria_tabela_auditoria.sql

```
CREATE TABLE auditoria_processo (
  id_auditoria BIGINT PRIMARY KEY,
  id_processo  BIGINT NOT NULL,
  operacao     VARCHAR(20) NOT NULL,
  usuario      VARCHAR(100) NOT NULL,
  data_evento  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT ck_operacao_aud
  CHECK (operacao IN ('INSERT', 'UPDATE', 'DELETE'))
);
```

A prova pode não citar Flyway ou Liquibase, mas pode apresentar exatamente a lógica de evolução estrutural. A categoria continua sendo DDL.

Transação e DDL: evite absolutismos. O comportamento de DDL dentro de transação varia entre SGBDs. O ponto seguro é: DDL define estrutura; TCL controla transação.

6. DML - Data Manipulation Language

DML - Data Manipulation Language - manipula dados armazenados em tabelas. É a linguagem do CRUD - Create, Read, Update and Delete - na perspectiva de banco: inserir, alterar e remover registros. SELECT costuma ser tratado separadamente como consulta, mas aparece junto do universo SQL.

Comando	Efeito
INSERT	Insere novas linhas.

Comando	Efeito
UPDATE	Atualiza linhas existentes.
DELETE	Remove linhas existentes.
MERGE	Combina inserção/atualização/remoção conforme correspondência entre origem e destino, se suportado pelo SGBD.

6.1 INSERT, UPDATE e DELETE

```
INSERT INTO servidor (matricula, nome, lotacao)
VALUES ('SC001', 'Ana Souza', 'Diretoria de Tecnologia');
```

```
UPDATE processo
SET segredo_justica = TRUE
WHERE id_processo = 1001;
```

```
DELETE FROM processo
WHERE data_baixa < DATE '2010-01-01';
```

Cuidado operacional: UPDATE ou DELETE sem WHERE pode atingir todas as linhas da tabela. Em prova, isso costuma aparecer como consequência de erro em script de manutenção.

6.2 MERGE e integração de dados

MERGE é comando de sincronização. A ideia é comparar uma origem com uma tabela de destino e executar ações conforme haja ou não correspondência. Em prova, pense em carga incremental, integração e atualização de cadastro.

```
MERGE INTO cadastro_destino d
USING carga_origem o
ON (d.cpf = o.cpf)
WHEN MATCHED THEN
  UPDATE SET d.nome = o.nome
WHEN NOT MATCHED THEN
  INSERT (cpf, nome) VALUES (o.cpf, o.nome);
```

6.3 DML e integridade: o comando pode falhar

Operação	Exemplo de falha
INSERT	Inserir processo com número já existente viola UNIQUE.
UPDATE	Atualizar status para valor fora do CHECK viola domínio.
DELETE	Excluir processo referenciado por movimentação pode violar FOREIGN KEY.
MERGE	Sincronização pode falhar se gerar duplicidade em chave única.

7. DCL - Data Control Language

DCL - Data Control Language - controla privilégios de acesso. Em ambiente corporativo e público, é tema sensível porque se conecta a segregação de funções, menor privilégio e auditoria.

```
GRANT SELECT ON processo TO perfil_consulta;
REVOKE INSERT ON processo FROM perfil_consulta;
```

Comando	Ideia
GRANT	Concede privilégio.
REVOKE	Revoga privilégio concedido.
DENY	No SQL Server, nega explicitamente uma permissão; não é universal SQL.

FGV costuma aproximar: GRANT/REVOKE não são DDL nem DML. Eles não criam tabela e não alteram linha de negócio; controlam autorização de acesso.

8. TCL - Transaction Control Language

TCL - Transaction Control Language - controla o encerramento ou pontos intermediários de uma transação. A Aula 6 aprofundará ACID, isolamento e concorrência; aqui o foco é reconhecer os comandos.

Comando	Função
COMMIT	Confirma definitivamente as alterações da transação.
ROLLBACK	Desfaz alterações ainda não confirmadas.
SAVEPOINT	Cria ponto intermediário para retorno parcial.
SET TRANSACTION	Ajusta características da transação, como nível de isolamento, conforme SGBD.

```
BEGIN;
UPDATE conta SET saldo = saldo - 100 WHERE id_conta = 10;
UPDATE conta SET saldo = saldo + 100 WHERE id_conta = 20;
COMMIT;
```

Pegadinha: ROLLBACK não desfaz aquilo que já foi confirmado por COMMIT. E DDL não vira TCL apenas porque algum SGBD pode realizar commit implícito em certas situações.

9. Constraints - restrições de integridade

Constraints são restrições declaradas no banco para proteger a integridade dos dados. Elas reduzem dependência exclusiva da aplicação e impedem inconsistências mesmo quando várias aplicações acessam o mesmo banco.

Restrição	O que garante	Erro comum
PRIMARY KEY	Identificação única e não nula.	Achar que toda PK precisa ser simples. Pode ser composta.
FOREIGN KEY	Referência válida a outra tabela.	Achar que FK nunca pode ser nula. Pode, se permitido.
UNIQUE	Não repetição de valores.	Confundir com PK. UNIQUE pode permitir nulos conforme SGBD.
NOT NULL	Obrigatoriedade de valor.	Achar que garante unicidade. Não garante.
CHECK	Condição lógica sobre valores.	Achar que substitui FK para relacionamento.
DEFAULT	Valor padrão quando não informado.	Achar que valida regra complexa de negócio.

9.1 Exemplo integrado

```
CREATE TABLE movimentacao (
  id_movimentacao INTEGER PRIMARY KEY,
  id_processo     INTEGER NOT NULL,
  tipo            VARCHAR(30) NOT NULL,
  data_movimento DATE NOT NULL DEFAULT CURRENT_DATE,
  visibilidade    VARCHAR(20) NOT NULL,
  CONSTRAINT fk_mov_processo
    FOREIGN KEY (id_processo) REFERENCES processo(id_processo),
  CONSTRAINT ck_visibilidade
    CHECK (visibilidade IN ('PUBLICA', 'RESTRITA', 'SIGILOSA'))
);
```

A questão pode perguntar qual restrição impede visibilidade inválida: é CHECK, não FOREIGN KEY. Pode perguntar qual restrição impede movimento sem processo existente: é FOREIGN KEY, não CHECK.

9.2 Escolha da ferramenta correta

Regra de negócio	Restrição mais provável
Todo processo deve ter número.	NOT NULL.
Número do processo não pode repetir.	UNIQUE ou PRIMARY KEY, conforme o desenho.
Movimentação deve pertencer a processo existente.	FOREIGN KEY.
Quantidade deve ser maior que zero.	CHECK.
Data de cadastro deve assumir a data corrente se omitida.	DEFAULT.
Cada item de pedido é identificado por pedido + produto.	PRIMARY KEY composta.

Regra de ouro: não use trigger mentalmente para tudo. Em prova, se a regra é declarativa e simples, constraint geralmente é a resposta mais limpa.

10. ON DELETE e efeito referencial

```
FOREIGN KEY (id_processo)
REFERENCES processo(id_processo)
ON DELETE RESTRICT;
```

Ação referencial	Consequência
RESTRICT / NO ACTION	Impede apagar o registro pai se houver filhos dependentes.
CASCADE	Propaga exclusão/atualização aos filhos.
SET NULL	Define a FK dos filhos como nula, se a coluna permitir.
SET DEFAULT	Define a FK com valor padrão, se válido.

Conexão com sistemas críticos: em dados bancários ou judiciais, CASCADE deve ser usado com extremo cuidado. Em prova, CASCADE é a ação que propaga, não a que bloqueia.

11. Views

View é um **objeto definido por uma consulta**. Ela apresenta uma relação lógica para o usuário, escondendo complexidade, restringindo colunas ou padronizando uma visão de dados. Em view comum, o resultado não é fisicamente materializado por padrão; a consulta é reavaliada quando a view é referenciada, conforme o SGBD.

```
CREATE VIEW vw_processos_publicos AS
SELECT id_processo, numero, classe, data_distribuicao
FROM processo
WHERE segredo_justica = FALSE;
```

Tipo/uso	Ideia
View comum	Consulta armazenada como objeto lógico; não materializa linhas por padrão.
View materializada	Armazena fisicamente o resultado e precisa ser atualizada/refrescada conforme o SGBD.
View de segurança	Expõe apenas colunas/linhas permitidas.
View de compatibilidade	Mantém interface estável enquanto tabelas internas evoluem.

Pegadinha: view não é sinônimo de tabela física. Também não é trigger, procedure ou índice. Ela encapsula uma consulta.

11.1 Views em segurança, compatibilidade e BI

```
CREATE VIEW vw_servidor_publico AS
SELECT matricula, nome, lotacao
FROM servidor
WHERE ativo = TRUE;
```

-- A view oculta colunas sensíveis como salário, CPF interno ou dados de contato.

Views aparecem em três usos típicos: segurança, compatibilidade e simplificação de consultas para relatórios ou BI - Business Intelligence.

12. Triggers

Trigger é um mecanismo que executa automaticamente uma rotina em resposta a evento no banco. Pode ser usada para auditoria, validação complementar, manutenção de histórico ou sincronização interna. Mas trigger em excesso pode esconder efeitos colaterais e dificultar depuração.

```
CREATE TRIGGER trg_audita_movimentacao
AFTER INSERT ON movimentacao
FOR EACH ROW
EXECUTE FUNCTION registra_auditoria_movimentacao();
```

Classificação	Exemplos
Por momento	BEFORE, AFTER, INSTEAD OF.

Classificação	Exemplos
Por granularidade	Por linha ou por comando, conforme SGBD.
Por evento	INSERT, UPDATE, DELETE; em alguns SGBDs também DDL ou logon.
Uso típico	Auditoria, histórico, validação, integração interna.

FGV gosta: trigger é automática e disparada por evento. Procedure é chamada explicitamente. View é consulta encapsulada. Constraint é regra declarativa.

12.1 Triggers: utilidade e risco

Uso adequado	Risco se mal usado
Auditoria de alterações sensíveis.	Efeito colateral oculto para a aplicação.
Manutenção de histórico técnico.	Dificuldade de debug e testes.
Validação complementar muito ligada ao dado.	Duplicação de regra com a aplicação.
INSTEAD OF trigger em view, quando suportado.	Confundir trigger com constraint simples.

13. Stored procedures

Stored procedure é uma rotina armazenada no banco, composta por uma ou mais instruções SQL e, conforme o SGBD, comandos procedurais. No SQL Server, procedures podem aceitar parâmetros de entrada e retornar valores de saída. Em Oracle, PostgreSQL, MySQL e SQL Server, a sintaxe varia, mas a ideia de encapsular lógica no banco é comum.

```
CREATE PROCEDURE baixa_processo(IN p_id_processo INTEGER, IN p_data_baixa DATE)
BEGIN
    UPDATE processo
    SET data_baixa = p_data_baixa
    WHERE id_processo = p_id_processo;
END;
```

Objeto	Chamado como?	Finalidade típica
View	SELECT sobre a view	Reutilizar consulta e controlar exposição de dados.
Trigger	Automaticamente por evento	Auditoria, validação, histórico.
Stored procedure	Chamada explicitamente	Encapsular operação ou rotina de banco.
Function	Chamada em expressão/consulta, conforme SGBD	Calcular e retornar valor ou tabela.

13.1 Utilidade e limites

Vantagem	Limite
Encapsula operação complexa no banco.	Pode dificultar portabilidade entre SGBDs.
Permite reuso por múltiplas aplicações.	Pode virar depósito de regra de negócio sem controle.
Pode receber parâmetros.	Sintaxe varia bastante entre MySQL, PostgreSQL, SQL Server e Oracle.
Pode participar de transações.	Não substitui automaticamente controle transacional correto.

14. Mini caso integrado

Imagine um sistema de acompanhamento processual. A equipe precisa criar tabela, validar status, permitir consulta pública, auditar alterações e controlar permissões.

```
CREATE TABLE processo (
    id_processo BIGINT PRIMARY KEY,
    numero VARCHAR(30) NOT NULL UNIQUE,
    status VARCHAR(20) NOT NULL CHECK (status IN ('ABERTO', 'SUSPENSO', 'ENCERRADO'))
);

CREATE VIEW vw_processo_publico AS
```

```
SELECT numero, status
FROM processo;
```

```
GRANT SELECT ON vw_processo_publico TO perfil_consulta_publica;
```

```
CREATE TRIGGER trg_audita_processo
AFTER UPDATE ON processo
FOR EACH ROW
EXECUTE FUNCTION audita_processo();
```

Leitura de prova: CREATE TABLE/VIEW/TRIGGER são DDL; CHECK e UNIQUE são constraints; GRANT é DCL; o trigger é disparado automaticamente; a view expõe consulta lógica; UPDATE sobre processo seria DML; COMMIT/ROLLBACK da operação seria TCL.

15. Quadro comparativo final

Item	Pergunta-chave	Resposta de prova
DDL	Mexe na estrutura?	Sim. CREATE, ALTER, DROP, TRUNCATE.
DML	Mexe em linhas de dados?	Sim. INSERT, UPDATE, DELETE, MERGE.
DCL	Mexe em permissões?	Sim. GRANT e REVOKE.
TCL	Mexe no destino da transação?	Sim. COMMIT, ROLLBACK e SAVEPOINT.
Constraint	Impõe regra declarativa?	Sim. PK, FK, UNIQUE, NOT NULL, CHECK, DEFAULT.
View	Guarda consulta?	Sim. Exibe resultado lógico; não materializa por padrão.
Trigger	Executa automaticamente por evento?	Sim.
Procedure	Rotina armazenada chamada explicitamente?	Sim.

16. Questões autorais estilo FGV - sem gabarito

Questão 1

No módulo de gestão processual de um tribunal, um analista precisa criar uma nova tabela para registrar os tipos de movimentação processual, definindo sua chave primária, uma descrição obrigatória e uma restrição para impedir códigos repetidos. Considerando as categorias clássicas da SQL - Structured Query Language -, a operação pertence predominantemente à categoria:

- A) DCL - Data Control Language, pois envolve concessão de privilégios sobre a tabela.
- B) DDL - Data Definition Language, pois define a estrutura de um objeto do banco de dados.
- C) DML - Data Manipulation Language, pois manipula registros processuais já existentes.
- D) TCL - Transaction Control Language, pois confirma a criação da tabela por meio de commit.
- E) DQL - Data Query Language, pois consulta metadados do catálogo antes da criação.

Questão 2

A equipe de sustentação executou o comando abaixo durante uma janela de manutenção.

```
TRUNCATE TABLE log_acesso;
```

Considerando o efeito típico desse comando em bancos relacionais, assinale a afirmativa correta.

- A) Remove a tabela log_acesso do catálogo, eliminando sua definição e seus privilégios.
- B) Remove todas ou quase todas as linhas da tabela, preservando sua estrutura, sem aplicar filtro WHERE.
- C) Remove apenas as linhas que atendem à última condição WHERE executada na sessão.
- D) Revoga os privilégios de escrita sobre a tabela, preservando os dados.
- E) Cria uma cópia vazia da tabela, mantendo a original intacta para rollback lógico.

Questão 3

Analise as afirmativas sobre DELETE, TRUNCATE e DROP.

- I. DELETE pode remover linhas específicas quando acompanhado de cláusula WHERE.
- II. DROP TABLE remove a estrutura da tabela do banco de dados.
- III. TRUNCATE TABLE é usado para alterar o tipo de uma coluna já existente.

Está correto o que se afirma em:

- A) I, apenas.
- B) II, apenas.
- C) I e II, apenas.
- D) II e III, apenas.
- E) I, II e III.

Questão 4

Um desenvolvedor deseja impedir que a coluna status de uma tabela assuma valores diferentes de ABERTO, SUSPENSO ou ENCERRADO. A forma declarativa mais adequada, no próprio banco relacional, é utilizar:

- A) uma constraint CHECK sobre a coluna status.
- B) uma view materializada contendo apenas os status válidos.
- C) um comando GRANT sobre a coluna status.
- D) um índice não único sobre a coluna status.
- E) um ROLLBACK automático ao final de cada transação.

Questão 5

No contexto de controle de acesso em banco de dados, relacione os comandos às respectivas finalidades.

- 1. GRANT
 - 2. REVOKE
 - 3. COMMIT
 - 4. SAVEPOINT
- () Conceder privilégio a usuário, papel ou perfil.
 - () Confirmar alterações de uma transação.
 - () Criar ponto intermediário de retorno em uma transação.
 - () Retirar privilégio anteriormente concedido.

A sequência correta é:

- A) 1 - 3 - 4 - 2.
- B) 1 - 4 - 3 - 2.
- C) 2 - 3 - 4 - 1.
- D) 3 - 1 - 2 - 4.
- E) 4 - 2 - 1 - 3.

Questão 6

Em uma rotina de importação, o script precisa inserir novos servidores e atualizar os existentes quando houver correspondência pela matrícula. Entre os comandos abaixo, aquele que melhor representa essa lógica de sincronização entre origem e destino, quando suportado pelo SGBD, é:

- A) MERGE.
- B) TRUNCATE.
- C) REVOKE.
- D) SAVEPOINT.
- E) DROP.

Questão 7

Considere o comando:

```
CREATE VIEW vw_processos_publicos AS
SELECT numero, classe, data_distribuicao
FROM processo
WHERE segredo_justica = FALSE;
```

À luz do conceito de view comum em banco relacional, assinale a afirmativa correta.

- A) A view cria necessariamente uma tabela física independente contendo cópia permanente dos dados.
- B) A view encapsula uma consulta e apresenta uma relação lógica derivada das tabelas subjacentes.
- C) A view é um gatilho executado automaticamente sempre que processo é atualizado.
- D) A view é uma constraint que impede processos sigilosos de serem gravados na tabela processo.
- E) A view é uma transação que confirma apenas consultas públicas.

Questão 8

Uma tabela MOVIMENTACAO possui a coluna id_processo, que deve referenciar PROCESSO(id_processo). A restrição mais apropriada para impedir movimentação associada a processo inexistente é:

- A) UNIQUE.
- B) CHECK.
- C) DEFAULT.
- D) FOREIGN KEY.
- E) NOT NULL.

Questão 9

Assinale a opção que contém apenas comandos usualmente associados à TCL - Transaction Control Language.

- A) CREATE, ALTER e DROP.
- B) INSERT, UPDATE e DELETE.
- C) GRANT, REVOKE e DENY.
- D) COMMIT, ROLLBACK e SAVEPOINT.
- E) SELECT, JOIN e ORDER BY.

Questão 10

Um administrador deseja permitir que o perfil RELATOR_CONSULTA leia a tabela PROCESSO, sem conceder permissão de escrita. O comando mais aderente a essa finalidade é:

- A) GRANT SELECT ON processo TO relator_consulta;
- B) ALTER TABLE processo ADD SELECT relator_consulta;
- C) UPDATE processo SET perfil = relator_consulta;
- D) COMMIT SELECT ON processo TO relator_consulta;
- E) CREATE VIEW relator_consulta AS GRANT processo;

Questão 11

A respeito de triggers, assinale V para a afirmativa verdadeira e F para a falsa.

- () Podem ser disparadas automaticamente em resposta a eventos como INSERT, UPDATE ou DELETE.
- () São sempre equivalentes a views, pois apenas armazenam consultas.
- () Podem ser usadas para auditoria, embora possam dificultar a depuração quando abusadas.

As afirmativas são, respectivamente:

- A) V, F e V.
- B) V, V e F.
- C) F, V e V.
- D) F, F e V.
- E) V, F e F.

Questão 12

Em uma base de dados corporativa, uma stored procedure foi criada para executar uma baixa processual com validações e atualizações em múltiplas tabelas. Em comparação com uma trigger, a diferença mais adequada é que a stored procedure:

- A) é chamada explicitamente, enquanto a trigger é disparada automaticamente por evento.
- B) não pode receber parâmetros, enquanto a trigger sempre recebe parâmetros nomeados.
- C) é sempre uma tabela virtual, enquanto a trigger é uma tabela física.
- D) é usada apenas para conceder privilégios, enquanto a trigger manipula dados.
- E) não pode conter comandos DML, enquanto a trigger pode conter apenas SELECT.

Questão 13

Considere o trecho:

```
CREATE TABLE item_pedido (  
    id_pedido INTEGER NOT NULL,  
    id_produto INTEGER NOT NULL,  
    quantidade INTEGER NOT NULL CHECK (quantidade > 0),  
    PRIMARY KEY (id_pedido, id_produto)  
);
```

A restrição CHECK declarada no comando tem por finalidade:

- A) impedir que dois itens tenham o mesmo par id_pedido e id_produto.
- B) garantir que id_produto exista previamente na tabela produto.
- C) impedir quantidade nula ou menor ou igual a zero.
- D) conceder permissão de atualização somente ao dono da tabela.
- E) confirmar automaticamente a transação após cada insert.

Questão 14

Uma equipe criou uma tabela com a coluna cpf definida como UNIQUE, mas não como PRIMARY KEY. Em termos conceituais, é correto afirmar que UNIQUE:

- A) garante, em regra, não repetição de valores, mas não transforma necessariamente a coluna em chave primária.
- B) é sinônimo absoluto de PRIMARY KEY em todos os SGBDs.
- C) garante apenas que a coluna não será nula, permitindo repetições.
- D) é comando TCL usado para preservar isolamento entre transações.
- E) só pode ser aplicada a colunas do tipo inteiro.

Questão 15

Em uma manutenção emergencial, foi executado o seguinte comando sem cláusula WHERE:

```
UPDATE servidor SET situacao = 'INATIVO';
```

O efeito lógico esperado é:

- A) alterar apenas o primeiro registro da tabela, conforme ordem da chave primária.
- B) alterar todas as linhas da tabela servidor, salvo restrição ou erro de execução.
- C) criar uma nova coluna situacao com valor INATIVO.
- D) revogar a permissão de ativação de servidores.
- E) desfazer automaticamente a transação por ausência de filtro.

Questão 16

A opção que apresenta corretamente uma associação entre categoria e exemplo de comando SQL é:

- A) DDL - INSERT INTO servidor VALUES (...).
- B) DML - GRANT SELECT ON processo TO usuario.
- C) DCL - ALTER TABLE processo ADD COLUMN segredo BOOLEAN.
- D) TCL - COMMIT.
- E) DDL - ROLLBACK TO SAVEPOINT sp1.

Questão 17

Uma view foi criada para ocultar a coluna salario de uma tabela de servidores e expor apenas matricula, nome e lotacao a determinado perfil. Nesse cenário, o uso de view está mais diretamente associado a:

- A) particionamento físico obrigatório dos dados.
- B) controle lógico de exposição de colunas e simplificação de consultas.
- C) execução automática de código após INSERT.
- D) confirmação parcial de transação.
- E) remoção definitiva da tabela original.

Questão 18

Em relação a DEFAULT, NOT NULL e CHECK, assinale a opção correta.

- A) DEFAULT impede que o usuário informe valor diferente do padrão.
- B) NOT NULL garante que o valor seja único em toda a tabela.
- C) CHECK permite expressar uma condição que os valores devem satisfazer.
- D) DEFAULT é uma categoria de comando equivalente a DCL.
- E) NOT NULL substitui FOREIGN KEY em relacionamentos entre tabelas.

Questão 19

No desenho de uma rotina de auditoria, deseja-se gravar automaticamente uma linha em AUDITORIA sempre que houver alteração em PROCESSO. O objeto de banco mais aderente a essa automação por evento é:

- A) view.
- B) trigger.

- C) constraint DEFAULT.
- D) role.
- E) savepoint.

Questão 20

Uma questão de prova afirma que DRL - Data Recovery Language - é uma categoria oficial clássica de comandos SQL, ao lado de DDL, DML, DCL e TCL. Essa afirmação deve ser considerada:

- A) correta, pois DRL é a categoria padrão de comandos SELECT.
- B) correta, pois DRL reúne GRANT e REVOKE.
- C) incorreta, pois a classificação clássica de prova reconhece DDL, DML, DCL e TCL, não DRL.
- D) incorreta apenas no PostgreSQL, mas correta no SQL Server 2019.
- E) correta apenas quando a tabela possui constraints de recuperação.

17. Gabarito resumido

1-B | 2-B | 3-C | 4-A | 5-A | 6-A | 7-B | 8-D | 9-D | 10-A | 11-A | 12-A | 13-C | 14-A | 15-B | 16-D | 17-B | 18-C | 19-B | 20-C

Observação editorial: o gabarito foi definido organicamente a partir da melhor alternativa de cada questão, sem distribuição matemática forçada de letras.

18. Comentários completos

Questão 1 - Gabarito: B

CREATE TABLE e criação de constraints são DDL, pois definem a estrutura do banco. A alternativa A seduz porque permissões também são SQL, mas DCL trata de GRANT/REVOKE. A C confunde estrutura com manipulação de linhas. A D confunde categoria do comando com confirmação transacional. A E traz DQL, mas a ação de criar tabela não é consulta.

Questão 2 - Gabarito: B

TRUNCATE esvazia a tabela preservando sua estrutura, usualmente sem WHERE. A alternativa A descreve DROP. A C inventa reaproveitamento de condição WHERE. A D descreve DCL. A E sugere cópia lógica, o que não é o efeito do comando.

Questão 3 - Gabarito: C

I está correta: DELETE pode filtrar linhas com WHERE. II está correta: DROP TABLE remove a estrutura da tabela. III está errada porque alterar tipo de coluna é ALTER TABLE, não TRUNCATE. A questão explora a tríade DELETE/TRUNCATE/DROP.

Questão 4 - Gabarito: A

CHECK expressa condição que limita valores admissíveis. View materializada não impede gravação inválida na tabela de origem. GRANT controla permissão, não domínio. Índice não único não valida conjunto de valores. ROLLBACK não é regra declarativa.

Questão 5 - Gabarito: A

GRANT concede privilégio; COMMIT confirma; SAVEPOINT cria ponto intermediário; REVOKE retira privilégio. As demais sequências trocam DCL com TCL, erro clássico de memorização.

Questão 6 - Gabarito: A

MERGE é o comando associado a sincronização entre origem e destino, com lógica de matched/not matched quando suportada. TRUNCATE esvazia tabela; REVOKE revoga permissão; SAVEPOINT cria ponto transacional; DROP remove objeto.

Questão 7 - Gabarito: B

View comum encapsula consulta e apresenta relação lógica derivada. A alternativa A descreve materialização obrigatória, o que não é regra para view comum. C descreve trigger. D descreve constraint/regra de integridade. E confunde view com transação.

Questão 8 - Gabarito: D

FOREIGN KEY garante que id_processo em MOVIMENTACAO referencie PROCESSO. UNIQUE evita repetição. CHECK valida condição local. DEFAULT fornece valor padrão. NOT NULL impede nulo, mas não garante existência do processo.

Questão 9 - Gabarito: D

COMMIT, ROLLBACK e SAVEPOINT são comandos típicos de controle transacional. A é DDL; B é DML; C é DCL ou extensão de permissão; E mistura consulta/cláusulas, não TCL.

Questão 10 - Gabarito: A

GRANT SELECT concede leitura. ALTER TABLE altera estrutura, UPDATE muda dados, COMMIT confirma transação e CREATE VIEW cria view. O comando correto precisa estar no universo DCL.

Questão 11 - Gabarito: A

A primeira afirmativa é verdadeira: triggers disparam por eventos. A segunda é falsa porque view armazena consulta; trigger executa rotina. A terceira é verdadeira: auditoria é uso típico, mas excesso prejudica

depuração.

Questão 12 - Gabarito: A

Stored procedure é normalmente chamada explicitamente; trigger dispara automaticamente por evento. Procedures podem receber parâmetros, então B erra. C confunde com view. D confunde com DCL. E erra porque procedures podem conter DML conforme SGBD.

Questão 13 - Gabarito: C

CHECK (quantidade > 0) impede quantidade menor ou igual a zero; NOT NULL impede nulo. A descreve PRIMARY KEY composta. B dependeria de FOREIGN KEY. D é DCL. E é TCL.

Questão 14 - Gabarito: A

UNIQUE garante não repetição, mas não é automaticamente PRIMARY KEY. PK agrega identificação oficial e não nulidade. B exagera equivalência. C inverte: NOT NULL impede nulo; UNIQUE lida com repetição. D e E fogem do conceito.

Questão 15 - Gabarito: B

UPDATE sem WHERE tenta atualizar todas as linhas. A alternativa A inventa limite pelo primeiro registro. C seria ALTER TABLE. D é DCL. E sugere rollback automático inexistente como regra geral.

Questão 16 - Gabarito: D

COMMIT é TCL. INSERT é DML, não DDL. GRANT é DCL, não DML. ALTER TABLE é DDL, não DCL. ROLLBACK é TCL, não DDL.

Questão 17 - Gabarito: B

View pode expor subconjunto de colunas e simplificar consultas. Não implica particionamento físico, não é automação por evento, não confirma transação e não remove a tabela original.

Questão 18 - Gabarito: C

CHECK expressa condição que deve ser satisfeita. DEFAULT fornece valor quando ausente, mas não impede outro valor válido. NOT NULL não garante unicidade. DEFAULT não é categoria DCL. NOT NULL não substitui FK.

Questão 19 - Gabarito: B

Trigger é o objeto adequado para ação automática em resposta a UPDATE/INSERT/DELETE. View apenas encapsula consulta; DEFAULT fornece valor; role agrupa permissões; savepoint é ponto de transação.

Questão 20 - Gabarito: C

Em classificação clássica de prova, DDL, DML, DCL e TCL são categorias reconhecidas; DRL - Data Recovery Language - não aparece como categoria oficial clássica. A e B redefinem DRL indevidamente. D cria diferença artificial entre SGBDs. E mistura categoria de comando com constraint.

19. Checklist final para a prova

- Saber classificar comandos: DDL estrutura; DML linhas; DCL permissões; TCL transações.
- Diferenciar DELETE, TRUNCATE e DROP sem hesitar.
- Reconhecer constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK e DEFAULT.
- Distinguir view comum, view materializada, trigger e stored procedure.
- Entender que trigger é automática por evento; procedure é chamada explicitamente.
- Lembrar que GRANT/REVOKE não manipulam dados de negócio; manipulam autorização.
- Lembrar que COMMIT confirma e ROLLBACK desfaz apenas o que ainda não foi confirmado.
- Em comandos UPDATE/DELETE, procurar WHERE: a ausência pode atingir a tabela inteira.

20. Referências técnicas usadas para calibragem

FGV - provas públicas correlatas com cobrança de categorias SQL, SQL em cenário de sistema e questões de banco de dados para cargos de TI.

Documentação oficial PostgreSQL: CREATE VIEW, CREATE TRIGGER, CREATE PROCEDURE e constraints.

Microsoft Learn: CREATE TRIGGER e stored procedures no SQL Server.

Documentação e materiais oficiais da Oracle/MySQL para noções de SQL e SGBD relacional.